Draft version October 14, 2026
Typeset using LaTeX **twocolumn** style in AASTeX7.0.1

Harmonica v2: Fixed-Work Transmission-String Light Curves for GPUs

Tyler Fairnington[1]

[1]*Department of Astronomy and Astrophysics, University of Chicago, Chicago, IL 60637, USA*

ABSTRACT

`harmonica` computes transit light curves of planets whose sky-projected silhouette is a Fourier series in angle, a "transmission string". Its C++ core finds the planet–star limb intersections from the eigenvalues of a companion matrix and integrates the occulted intensity with Green's theorem, branching on the geometry of each point. That structure is fast on one CPU core but maps poorly onto GPUs. Here I present a reformulation in which every light-curve point performs the same, statically shaped work, so that the model compiles with XLA, runs on GPUs, batches over many light curves, and composes with automatic differentiation, including to higher order. I bracket the intersections on angular grids, with a closed form for circles and a proven two-extremum bracket for first-order strings, and take their derivatives from the implicit-function theorem. I then assemble the occulted region from the direction in which the limbs cross, which I show handles tangencies and stellar arcs that wrap around the star. Against independent brute-force overlap areas and analytic cases, the new model agrees with the C++ to $\sim 10^{-10}$ at the default quadrature order and to $\sim 4 \times 10^{-15}$ at matched order, wherever the C++ is itself correct. Critically, this validation also uncovered several errors in the original implementation, including fluxes above unity for planets that cover the stellar centre and a flattened orbit near apastron. The new model is therefore the default on every platform, while an opt-in mode runs the faster but unchanged C++ on CPUs, now with correct higher derivatives.

Keywords: Exoplanet atmospheres (487) — Transits (1711) — Astronomy software (1855) — GPU computing (1969)

1. INTRODUCTION

Transmission strings describe the radius of a transiting planet as a function of sky-plane angle θ about its centre,

$$r_{\rm p}(\theta) = a_0 + \sum_{n=1}^{N_c} \left[a_n \cos n\theta + b_n \sin n\theta \right], \qquad (1)$$

so that the light curve constrains how the atmospheric opacity varies around the terminator (D. Grant & H. R. Wakeford 2023). The order N_c sets the angular resolution: $N_c = 0$ is a circle, while $N_c = 1$ adds a first-order asymmetry, for example between the morning and evening limbs. Recent JWST analyses use strings of order six (L. Alderson et al. 2026).

Inference with these models increasingly involves many light curves at once, e.g. hundreds of spectroscopic channels, cross-validation folds or ensemble samplers. GPUs suit this workload, but only for programs whose control flow and array shapes do not depend on the data. The original `harmonica` evaluates each point with a data-dependent sequence of branches, an eigenvalue problem whose number of real roots varies, and a quadrature order that switches with position. Simply wrapping it for a GPU therefore would not help: the algorithm itself has to change. In what follows I describe those changes, which I release on the `jax-gpu` branch of `harmonica`.[2]

2. THE MODEL

I follow the formulation of D. Grant & H. R. Wakeford (2023). For limb-darkening laws that are sums of powers of μ, $I(\mu) = \sum_n p_n \mu^n$, the normalised flux is

$$F = 1 - I_0 \sum_n p_n \, s_n, \qquad (2)$$

where s_n is the occulted integral of μ^n. By Green's theorem, each s_n is a line integral around the boundary of the occulted region, which consists of arcs of the planet's limb and of the stellar limb. In planet-centred polar coordinates, with the planet at separation d and angle ν

[2] https://github.com/TylerFair/harmonica/tree/jax-gpu

from the stellar centre, a planet-limb arc contributes

$$s_n^{(p)} = \int_{\theta_j}^{\theta_{j+1}} \zeta_n(z_\mathrm{p})\,\eta(\theta)\,\mathrm{d}\theta, \tag{3}$$

with

$$\eta = r_\mathrm{p}^2 - d\,r_\mathrm{p}\cos(\theta-\nu) - d\,r_\mathrm{p}'\sin(\theta-\nu), \tag{4}$$

$$z_\mathrm{p}^2 = 1 - d^2 - r_\mathrm{p}^2 + 2d\,r_\mathrm{p}\cos(\theta-\nu), \tag{5}$$

$$\zeta_n(z) = \frac{1-z^{n+2}}{(n+2)(1-z^2)}. \tag{6}$$

Here z_p is μ on the planet's limb. A stellar-limb arc, on which $\mu = 0$, contributes $\Delta\phi/(n+2)$, where $\Delta\phi$ is the arc's angular extent about the stellar centre.

The quadratic law has $p = (1-u_1-u_2,\ u_1+2u_2,\ -u_2)$ for $\mu^{0,1,2}$ and $I_0^{-1} = \pi(1-u_1/3-u_2/6)$. The non-linear law has $p = (1-\sum_i u_i,\ u_1, u_2, u_3, u_4)$ for $\mu^{0,1/2,1,3/2,2}$ and $I_0^{-1} = \pi(1-u_1/5-u_2/3-3u_3/7-u_4/2)$. I also add the power-2 law $I = 1-c(1-\mu^\alpha)$ (D. Hestroffer 1997; P. F. L. Maxted 2018), with $p = (1-c,\ c)$ for $\mu^{0,\alpha}$ and $I_0^{-1} = \pi[1-c+2c/(\alpha+2)]$. With $\alpha = 2$ it reduces exactly to the quadratic law with $u_1 = 2c$, $u_2 = -c$.

3. A FIXED-WORK FORMULATION

3.1. *Limb intersections*

The limbs intersect where

$$f(\theta) \equiv r_\mathrm{p}^2 + d^2 - 2d\,r_\mathrm{p}\cos(\theta-\nu) - 1 = 0, \tag{7}$$

with $f < 0$ inside the star. Since r_p is a trigonometric polynomial of degree N_c, f is one of degree $D = \max(2N_c, N_c+1)$, and has at most $2D$ real roots on the circle. The C++ finds them as eigenvalues of a $2D \times 2D$ companion matrix, which requires a_{N_c} and b_{N_c} not both to vanish. Instead, I use three bracketing schemes, each followed by bisection.

Circles. When every a_n, b_n $(n \geq 1)$ is zero, the roots are $\nu \pm \beta$, with

$$\beta = \mathrm{atan2}\left(\sqrt{\Delta},\ d^2 + a_0^2 - 1\right), \tag{8}$$

where $\Delta = [1-(d-a_0)^2][(d+a_0)^2-1]$. The factored discriminant stays accurate at contact, where the usual arccos form loses half its digits.

First-order strings $(N_c = 1)$. Here f contains only the first and second harmonics,

$$f = c_0 + A\cos(\theta-\phi) + B\cos(2\theta-\psi), \tag{9}$$

with $A\cos\phi = 2a_0(a_1 - d\cos\nu)$, $A\sin\phi = 2a_0(b_1 - d\sin\nu)$, $B\cos\psi = (a_1^2-b_1^2)/2 - d(a_1\cos\nu - b_1\sin\nu)$ and $B\sin\psi = a_1b_1 - d(a_1\sin\nu + b_1\cos\nu)$.

Proposition. If $A > \sqrt{20}\,B$, then f has exactly one maximum in $(\phi-\pi/2, \phi+\pi/2)$ and one minimum in $(\phi+\pi/2, \phi+3\pi/2)$, and no other stationary points.

Proof. Let $x = \theta - \phi$. Then $f' = -A\sin x - 2B\sin(2\theta-\psi)$ vanishes only where $|\sin x| \leq 2B/A$, and there $|\cos x| \geq (1-4B^2/A^2)^{1/2}$. Hence on that set $|f''| \geq A(1-4B^2/A^2)^{1/2} - 4B$, which is positive exactly when $A^2 > 20B^2$. So f' is strictly monotone on each of the two arcs on which it can vanish. Since $2B < A$, $f'(\phi-\pi/2) > 0 > f'(\phi+\pi/2)$, and similarly with the signs exchanged at $\phi+\pi/2$ and $\phi+3\pi/2$, so each arc contains exactly one zero. □

When the condition holds, which it does for any physically small asymmetry, I bisect both extrema on these certified brackets. Roots exist if and only if f is positive at the maximum and negative at the minimum, and one lies on each arc between them, on which f is monotone. This finds a pair of intersections arbitrarily close to contact, which a fixed angular grid can miss entirely.

General strings. Otherwise, including for uncertified $N_c = 1$ shapes, I evaluate f and f' on a uniform grid of $G = \max(128, 64N_c)$ intervals of width $h = 2\pi/G$, bisect every sign change of f', and insert the resulting extrema into the grid. Sign changes of f between consecutive nodes then bracket the roots. Because the nodes form a closed loop, I give any node at which f is exactly zero the sign of the preceding node, so that a crossing on a node counts once and a tangency, which does not change sign, does not count at all. A closed curve then crosses an even number of times, at most $2D$. Any other count means the bracketing failed, and the model returns NaN rather than a plausible but wrong flux.

This scheme is not certified, however. It finds every transverse root if each grid interval contains at most one zero of f', e.g. if distinct zeros of f' are separated by more than h. If $|f'''| \leq L$ and $|f''| \geq m > 0$ at every critical point, Taylor's theorem gives a separation of at least $2m/L$, so $h < 2m/L$ suffices. A computable per-cell alternative, with midpoint c and half-width $q = h/2$, is that either $|f''(c)| > Lq$, so that f' is monotone on the cell, or $|f'(c)| > |f''(c)|\,q + Lq^2/2$, so that it has no zero there. Writing $A_n = (a_n^2+b_n^2)^{1/2}$, $R_0 = |a_0| + \sum_n A_n$ and $R_j = \sum_n n^j A_n$, a conservative bound is $L = 6R_1R_2 + 2R_0R_3 + 2|d|(R_3 + 3R_2 + 3R_1 + R_0)$. A cell that fails both tests is uncertified, not necessarily wrong.

3.2. *Root derivatives*

Differentiating through bisection gives derivatives of zero, so I instead give the root finder a custom derivative rule from the implicit-function theorem, $\dot\theta_j = -\dot f/f_\theta$,

with

$$f_\theta = 2r_\mathrm{p}r'_\mathrm{p} - 2d\left(r'_\mathrm{p}C - r_\mathrm{p}S\right), \tag{10}$$

$$\dot{f} = 2\left(r_\mathrm{p} - dC\right)\dot{r}_\mathrm{p} + 2\left(d - r_\mathrm{p}C\right)\dot{d} - 2d\,r_\mathrm{p}S\,\dot{\nu}, \tag{11}$$

with $C = \cos(\theta - \nu)$ and $S = \sin(\theta - \nu)$, and $\dot{r}_\mathrm{p}$ is Equation (1) evaluated with the coefficient tangents. Padded roots, and roots with $|f_\theta| < 10^{-14}$, receive zero tangents. I make the denominator safe *before* dividing, since masking a 0/0 afterwards still leaks NaN into reverse-mode derivatives. JAX then differentiates everything downstream of the roots automatically.

3.3. *Assembling the occulted region*

The C++ classifies each pair of consecutive crossings with a lookup table of crossing types, and integrates the stellar arc between them using the principal difference of their stellar angles. As I show in Section 6, this misassembles some configurations, so I instead build the boundary from the direction in which the limbs cross.

Planet arcs. Order the crossings by θ. Since $f < 0$ inside the star, the planet-limb arc after a crossing bounds the occulted region exactly when $f_\theta < 0$ there.

Stellar arcs. Order the crossings instead by their angle ϕ about the stellar centre, and take each arc anticlockwise to the next, wrapping the last by 2π. The arc after a crossing bounds the region exactly when $f_\theta > 0$ there. To see this, parametrise the stellar limb by ϕ, with $\rho(\phi)$ and $\theta(\phi)$ the distance and angle about the planet's centre, and let $H(\phi) = r_\mathrm{p}^2(\theta(\phi)) - \rho^2(\phi)$, which is positive inside the planet. At a crossing, $\theta_\phi = (1 - d\cos\phi)/r_\mathrm{p}^2$, $\rho_\phi = d\sin\phi/r_\mathrm{p}$, and $1 - d^2 = r_\mathrm{p}^2 - 2d\,r_\mathrm{p}\cos(\theta - \nu)$, from which $H_\phi = f_\theta$. H cannot change sign before the next transverse crossing, and tangencies omitted from the root list do not change it.

This construction needs positive radii and a complete list of transverse crossings, but no midpoint tests, crossing-type tolerances or angle-wrapping heuristics. Every point evaluates $K = 2D$ potential arcs of each kind, masked by these conditions, and I give masked arcs benign geometry so that they cannot produce non-finite derivatives.

I resolve configurations without crossings in a fixed order. Bounds $r_\mathrm{p} \in [a_0 - \sum_n A_n,\, a_0 + \sum_n A_n]$, with a 10^{-12} tolerance, identify planets wholly inside or outside the star and stars wholly inside the planet; they are exact for $N_c \leq 1$ and conservative otherwise. Such a trivial classification overrides any roots and integrates the entire limb; otherwise, a root pair within the tolerance of contact would integrate over only its tiny connecting arc. With no roots otherwise, the limb point facing the stellar centre, $r_\mathrm{p}(\nu)$, decides between the remaining cases.

3.4. *Line integrals*

The C++ evaluates s_0 and s_2 in closed form and the other terms by Gauss–Legendre quadrature, of order 20 or 50 depending on position. I apply a single order n_GL (default 50) to every planet-limb term, which removes the position-dependent branch and is at least as accurate as the C++ default (Table 1).

Equation (6) has a removable singularity at $z = 1$, the stellar disc centre, and cancels catastrophically near it. I use exact rearrangements that avoid both:

$$\zeta_0 = \tfrac{1}{2}, \qquad \zeta_2 = \tfrac{1}{4}(1 + z^2), \tag{12}$$

$$\zeta_1 = \tfrac{1}{3}\left[z + (1 + z)^{-1}\right], \tag{13}$$

$$\zeta_{1/2} = \frac{2}{5}\,\frac{\sum_{k=0}^{4} s^k}{(1 + s)(1 + s^2)}, \tag{14}$$

$$\zeta_{3/2} = \frac{2}{7}\,\frac{\sum_{k=0}^{6} s^k}{(1 + s)(1 + s^2)}, \tag{15}$$

where $s = z^{1/2}$. For the power-2 term, with $t = 1 - z^2$ and $p = (\alpha + 2)/2$, I sum twelve terms of the Taylor series of $[1 - (1 - t)^p]/(2pt)$ when $t < \min(0.01, 0.1/p)$, which keeps its truncation error below $\sim 10^{-21}$ for any α; evaluate

$$\zeta_\alpha = -\frac{\mathrm{expm1}[p\ \mathrm{log1p}(-t)]}{2pt} \tag{16}$$

for larger $t < 1/2$; and use Equation (6) directly beyond. Against a 75-digit reference for α from 0.01 to 100 and t from 0 to 0.49, the values agree to 6×10^{-16}, and the first and second derivatives to 2×10^{-14} and 4×10^{-12}. No finite series, of course, supplies correct derivatives of arbitrarily high order.

3.5. *Orbits*

The orbit follows the C++, including its rotation of the string with the orbit, but with three changes. First, I solve Kepler's equation on $[0, \pi]$ by Newton iterations safeguarded by bisection, on $(1 - e)E + e(E - \sin E) = M$ with a series for $E - \sin E$ near zero, so that it does not cancel as $e \to 1$, and take its derivatives from the implicit-function theorem. Second, the true anomaly and radius follow from $s = \sin(E/2)$, as $\cos f = [(1 - e) - 2s^2]/[(1 - e) + 2es^2]$, $\sin f = \sqrt{(1 - e)(1 + e)}\,\sin E/[(1 - e) + 2es^2]$ and $r = a[(1 - e) + 2es^2]$, which avoid both cancellation near $e = 1$ and the singular half-angle formula, $\tan(f/2) \propto \tan(E/2)$, near apastron. Third, the eccentric anomaly at transit is $E_0 = \mathrm{atan2}(\sqrt{(1 - e)(1 + e)}\cos\omega,\, e + \sin\omega)$, whose derivatives are regular at $\omega = -\pi/2$. When e is a compile-time zero, the model skips Kepler's equation entirely; when it is traced, it always uses the eccentric solution, so that $\partial F/\partial e$ stays correct at $e = 0$.

3.6. *Derivatives and the CPU path*

On a single CPU core, padding every point to the most complex geometry costs about an order of magnitude relative to the C++. Because the C++ still contains the errors in Section 6, however, the public functions use the fixed-work model on every platform by default. With `backend='auto'`, they instead choose an implementation per platform at compile time (`jax.lax.platform_dependent`): the C++ on CPUs, and the fixed-work model everywhere else. To make this possible, I moved the C++ JAX primitives to JAX's public foreign function interface, as JAX has since removed the APIs they used. The C++ also crashes outright on strings whose final two coefficients are both zero, so the wrapper refuses those, and the `'auto'` mode sends them to the fixed-work model. The C++ supplies only first derivatives, so I give its Jacobian its own derivative rule, the fixed-work model's Hessian-vector product. JAX traces that rule only for second and higher derivatives: compiled first-order programs contain no fixed-work evaluations and keep the C++'s speed. The flux depends on time only through $t - t_0$, which gives the time derivatives on this path.

More fundamentally, automatic differentiation gives exact derivatives of the implemented function. These approximate the model's derivatives only to the accuracy of the quadrature and series above, and only where the topology is resolved. At a generic contact with a limb-darkened star, the occulted flux grows locally as $\delta^{3/2}$ in the penetration depth δ, so the model's second derivatives are themselves singular there, and finite derivatives assigned at contact or tangency (e.g. the $|f_\theta|$ guard) do not make it smooth.

4. VALIDATION

All of my tests use double precision, which the model requires. Agreement with the C++ alone cannot establish correctness (Section 6), so I lead with independent references.

- *Overlap areas.* For a uniform star, $1 - F$ is the overlap area over π, which I integrate independently, in closed form along rays from the planet's centre. Over 6000 random positive-radius shapes, N_c = 1–4, a_0 = 0.2–1.2 and d anywhere, including planets that cover the stellar centre and multi-lobed overlaps, the largest flux difference is 5×10^{-8}; such differences come from the reference itself, and every one above 10^{-8} falls below 2×10^{-10} when I raise its resolution from 2×10^5 to 8×10^6 rays.

- *Analytic cases.* The shape $r_\mathrm{p} = 1 + a\cos\theta - a\cos^2\theta$ centred on the star has crossings at $\pm\pi/2$ and a tangency at $\theta = 0$, and $F = a/2 + 2a/\pi - 7a^2/16 - 4a^2/(3\pi)$, which the model reproduces to 10^{-15}. It also reproduces a tangency coexisting with crossings, $r_\mathrm{p} = 0.125 - 0.00390625\cos 6\theta$ at $d = 1.12109375$ ($F = 0.999999975014641$), and nearby four-crossing cases.

- *C++ parity.* For N_c = 0–3, circular and eccentric orbits, and both C++ laws, the default order agrees to $\leq 10^{-10}$, and matched quadrature to $\sim 4 \times 10^{-15}$ (Table 1), in configurations where the C++ is correct. Power-2 also agrees with a separate C++ implementation of the law to 1.2×10^{-10} for $N_c \leq 2$; as that requires a different build, this one check is not in `paper/reproduce`.

- *Derivatives.* Reverse- and forward-mode derivatives agree to 3×10^{-14}, and $\partial F/\partial e$ at $e = 0$ agrees with finite differences to 7×10^{-9}. They agree with the C++ analytic derivatives to $\leq 10^{-5}$, with the largest differences just inside contact, where mine agree with converged finite differences to $\sim 10^{-10}$ (Section 6). Through the CPU path, the Hessian of a planet inside a uniform star is -2 as required, and partial-transit Hessians match the fixed-work model's to 5×10^{-15}.

- *Orbits.* The derivative of $\sin f$ at apastron is exact, and the true anomaly at $e = 0.99999999$ near periastron matches a long-double reference to 10^{-8}.

- *Inference.* Re-running the `harmonica` gradient-based inference tutorial (N_c = 3, NUTS via NumPyro; D. Phan et al. 2019) reproduces its published posterior within sampling noise, with $\hat{R} \leq 1.004$.

5. PERFORMANCE

Table 2 compares the fixed-work model on an otherwise idle NVIDIA V100 with the C++ on one core of the same node. The GPU gains grow with the string's order and the number of points, from 1.5 times for a first-order string at 10^3 points to 9 times at $N_c = 3$ and 10^4 points, where a value and gradient take 14.5 ms. On this card, the harmonica gradient-based inference tutorial (NUTS, $N_c = 3$, two chains of 700 iterations) takes 62 s including compilation.

For a workload like that of L. Alderson et al. (2026), order-six cosine-only strings and 582 points at a 60 s cadence, the C++ takes 27 ms per light curve on one core of the same node. The fixed-work model takes 1.4 ms for

Table 1. Maximum flux error against 500-point quadrature

Quadrature	Max. $\lvert\Delta F\rvert$
$n_{\mathrm{GL}} = 20$	1×10^{-9} – 7.5×10^{-9}
$n_{\mathrm{GL}} = 50$ (default)	1.1×10^{-11} – 9.2×10^{-11}
$n_{\mathrm{GL}} = 100$	3.6×10^{-13} – 4.2×10^{-12}
$n_{\mathrm{GL}} = 200$	1.1×10^{-14} – 2.6×10^{-13}
$n_{\mathrm{GL}} = 500$	2.8×10^{-15} – 1.5×10^{-14}
C++ default (20/50)	1.1×10^{-11} – 7.6×10^{-10}

NOTE—Ranges span the quadratic and non-linear laws and $N_c = 0$–3, the maximum over four light curves each. In one quadratic $N_c = 1$ case, orders 200 and 500 both differ by 2.6×10^{-13}, which is the reference's own error. The largest errors occur for planets just inside the stellar limb, where the integrand has a square-root endpoint.

Table 2. Time per light curve (ms)

Points	N_c	C++, 1 core	GPU	GPU, value+grad
10^3	1	3.0	2.1	2.2
10^3	2	6.6	1.5	1.7
10^3	3	11.2	1.6	2.3
10^4	1	30.7	14.3	14.9
10^4	2	63.3	7.5	9.0
10^4	3	107.6	11.8	14.5

NOTE—GPU: an otherwise idle NVIDIA V100. The C++ column is the flux only, without derivatives. First-order strings are slower than second-order ones here because every point evaluates both the certified bracket and the grid fallback of Section 3.1.

one light curve, and 0.72 ms each in batches of 1560 or more, about 37 cores' worth; it evaluates large batches in chunks of bounded memory. Their cross-validated ensemble fit of one wavelength bin ($26 \times 5000 \times 10 \times 12 \approx 1.6 \times 10^7$ light curves) would therefore need ∼116 core-hours of C++ evaluations, or ∼3.1 hours on one V100. They report about two weeks on 40 cores per bin, which these kernel costs alone do not account for; they do not, however, establish where that time was spent. Only a matched, end-to-end comparison of complete fits could show the practical speedup.

6. ISSUES FOUND IN THE ORIGINAL IMPLEMENTATION

Validation turned up the following problems in the C++ and its JAX interface, all of which v2's fixed-work model avoids. I have not changed the C++ itself.

1. For planets that cover the stellar centre, a stellar arc crossing the far side of the star loses 2π in its principal angle difference, so the occulted integral can be wrong by a whole star. Fluxes above unity result, e.g. 1.0003 for an $a_0 = 1.5$ string whose true flux is 0.0003.

2. The half-angle conversion to the true anomaly clamps $\sin f = 0$, $\cos f = -1$ whenever $1 + \cos E \leq 10^{-10}$, which flattens the orbit near apastron ($\partial \sin f / \partial M = 0$ instead of -0.82 at $e = 0.1$).

3. The eccentric anomaly at transit uses the half-angle form, whose derivatives with respect to ω are wrong near $\omega = -\pi/2$.

4. At $e = 0$, $\partial F / \partial e$ does not depend on ω (e.g. 6×10^{-4} against a true 0.086).

5. The derivative of a circular planet's flux with respect to a_0 is in error by $\sim 2 \times 10^{-6}$ just inside the second contact.

6. When an endpoint combination is absent from the segment lookup table, the code records no segment type, which misaligns the types of the following segments (which I found by reading the code).

7. The JAX interface does not import on JAX ≥ 0.6, and it broadcast each orbital and limb-darkening parameter to the times while the C++ reads only the first element, silently truncating per-time arrays. I fix both in v2.

8. The C++ crashes the Python process on strings whose final two coefficients are both zero, e.g. a zero-padded circle, rather than raising an error.

7. SUMMARY

The changes in `harmonica` v2 are algorithmic, not a new wrapper around the C++. Bracketing replaces eigenvalues for the intersections, the implicit-function theorem supplies their derivatives, the directions of the crossings assemble the occulted region, and rewritten radial and orbital formulae avoid cancellation. Together, these give every light-curve point the same static work, which XLA can compile for GPUs, batch and differentiate. The release supports any N_c, the quadratic, non-linear and power-2 laws, and can run the C++ on CPUs on request. The code is built on JAX (J. Bradbury et al. 2018), and the scripts and saved outputs behind every number in Sections 4 and 5 are in `paper/reproduce` on the `paper-v2` branch.

ACKNOWLEDGMENTS

I thank D. Grant for developing `harmonica` and making it public, and a colleague for a careful review that found several of the issues I correct here.

Software: `harmonica` (D. Grant & H. R. Wakeford 2023), JAX (J. Bradbury et al. 2018), NumPyro (D. Phan et al. 2019)

REFERENCES

Alderson, L., Wakeford, H. R., Grant, D., et al. 2026, arXiv e-prints. https://arxiv.org/abs/2610.18806

Bradbury, J., Frostig, R., Hawkins, P., et al. 2018, JAX: composable transformations of Python+NumPy programs, http://github.com/google/jax

Grant, D., & Wakeford, H. R. 2023, MNRAS, doi: 10.1093/mnras/stac3632

Hestroffer, D. 1997, A&A, 327, 199

Maxted, P. F. L. 2018, A&A, 616, A39, doi: 10.1051/0004-6361/201832944

Phan, D., Pradhan, N., & Jankowiak, M. 2019, arXiv e-prints. https://arxiv.org/abs/1912.11554